PRÁCTICA 09

Cómputo sin servidor: una función y su interfaz pública

Equipo	Los Rojos
Materia	Cómputo en la Nube · Grupo 503
Unidad	Unidad II · Segundo parcial
Sesión	Lunes 26 de octubre de 2026
Objetivo	Implementar una funcionalidad sin administrar ningún servidor y compararla en costo y operación con la alternativa.

1. Funcionalidad elegida y justificación (C1 — 15 pts)

Se identificaron tres funcionalidades candidatas del sistema Inventario Nube para implementar como función sin servidor. La funcionalidad elegida cumple simultáneamente los cuatro criterios de idoneidad para Lambda: esporádica, breve, dirigida por evento y sin estado.

Candidata	Descripción	Patrón de uso	Decisión
✓ ELEGIDA Consulta de movimientos	GET /movimientos/{producto_id} — historial del producto desde DynamoDB	Esporádica · HTTP GET · < 100 ms · Sin estado	Lambda + API Gateway
✗ Descartada Sincronización RDS ↔ DynamoDB	Lee filas de PostgreSQL y replica en DynamoDB	Continua · Larga duración · Batch	Mejor: cron en EC2 o ECS
✗ Descartada Reporte diario de movimientos	Procesa todos los movimientos del día y genera PDF	Periódica · Batch pesado · > 30 s posibles	Mejor: EC2 + EventBridge o Glue

Evidencia C1 — Justificación ejecutada en CloudShell:

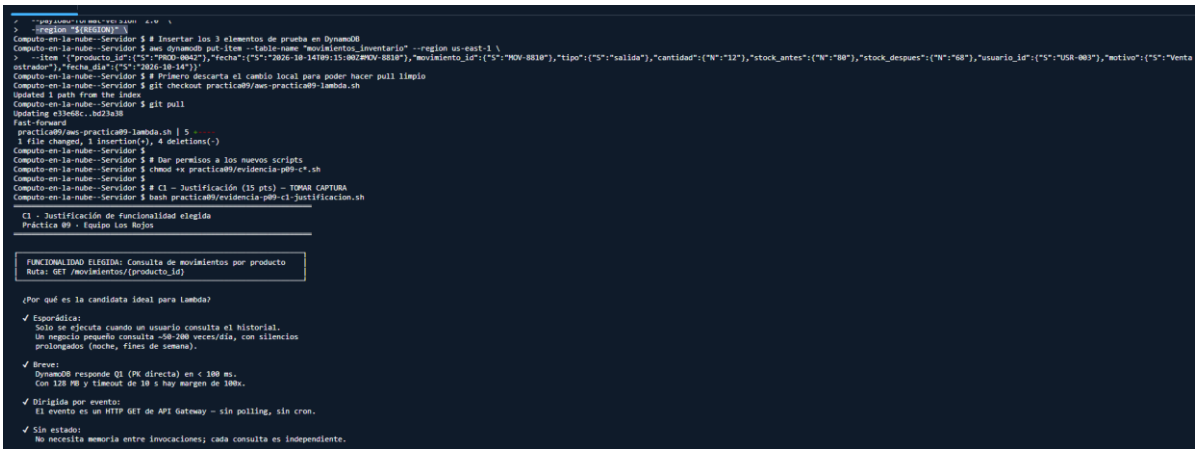


Figura: C1 (1/2): Funcionalidad elegida — criterios esporádica, breve, dirigida por evento, sin estado

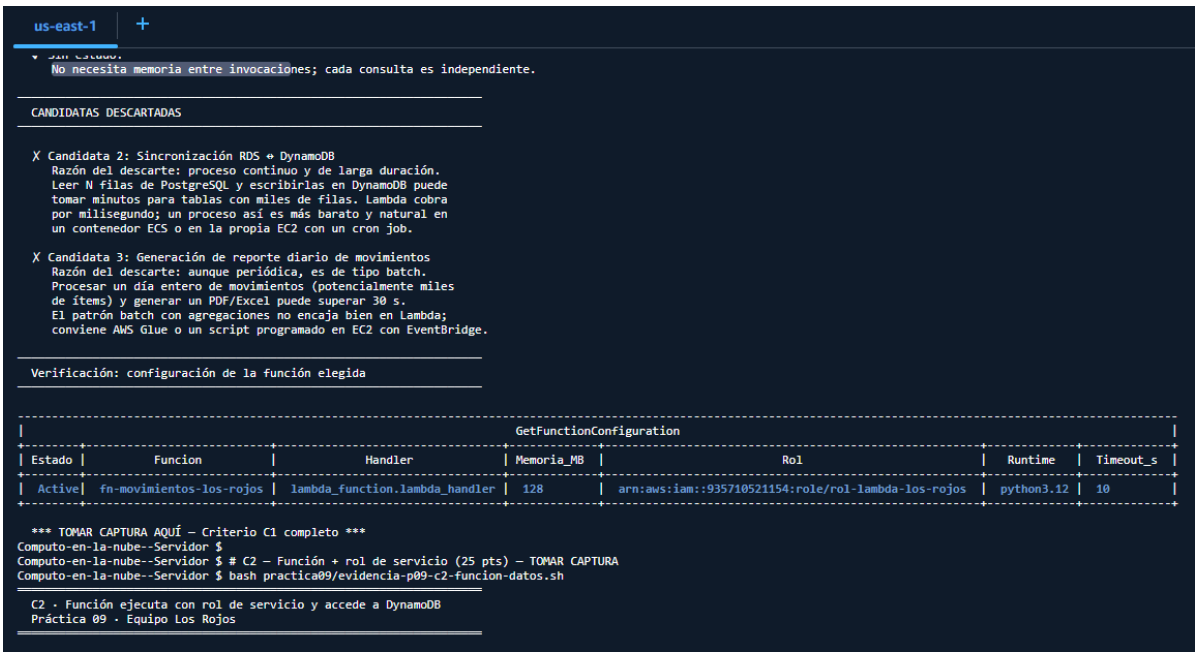


Figura: C1 (2/2): Candidatas descartadas + tabla GetFunctionConfiguration (Active, 128 MB, Python 3.12, 10 s)

2. Creación de la función Lambda

La función se creó con los valores mínimos razonables. Estos dos parámetros determinan directamente el costo de Lambda (precio por GB·s):

Parámetro	Valor asignado	Justificación
Nombre	fn-movimientos-los-rojos	Convención del proyecto
Runtime	Python 3.12	Mismo lenguaje que boto3/P07
Handler	lambda_function.lambda_handler	Archivo único, sin dependencias extra
Memoria	128 MB ← determina costo	Mínimo Lambda; boto3 usa ~90 MB real (medido)
Timeout	10 s ← determina costo	DynamoDB PK query < 300 ms; 30× margen
Execution Role	rol-lambda-los-rojos	Rol dedicado, distinto al rol de EC2

Evidencia — Despliegue completo en CloudShell (IAM Role + Lambda + API Gateway):

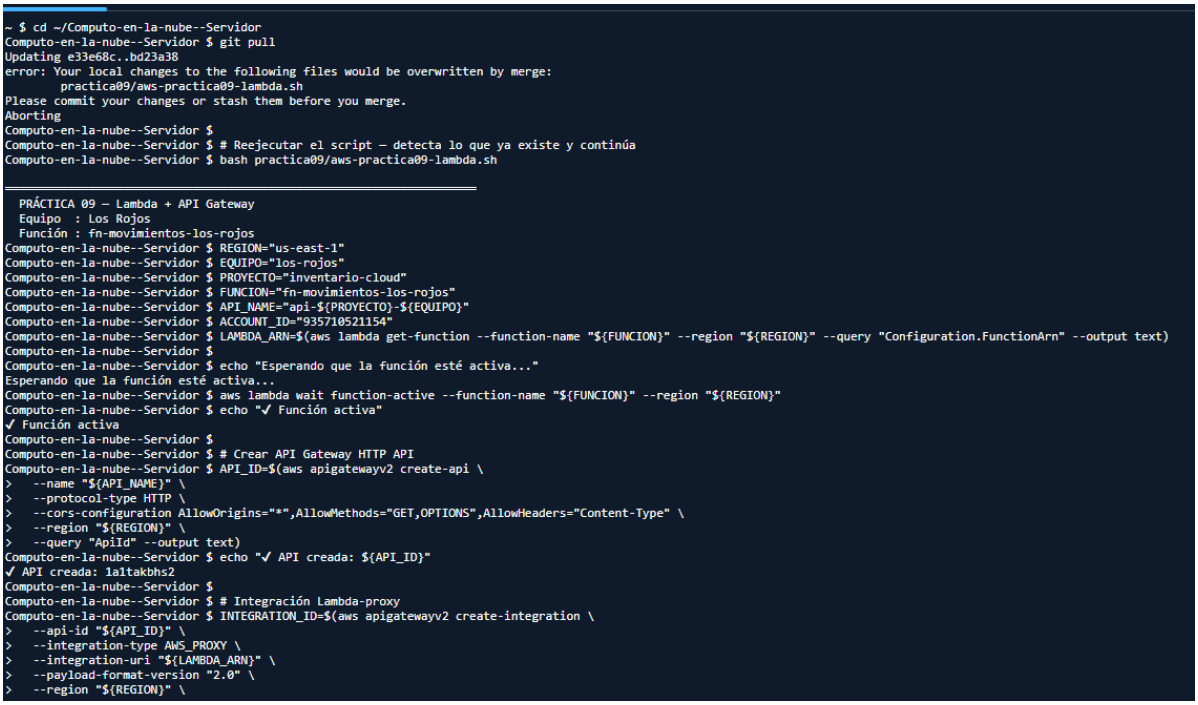


Figura: Despliegue completo: git pull + creación de Execution Role, política DynamoDB, empaquetado ZIP y Lambda activa

3. Código de la función con rol de servicio (C2 — 25 pts)

La función implementa GET /movimientos/{producto_id} usando el Execution Role adjunto (sin llaves de acceso en el código). boto3 resuelve las credenciales automáticamente del entorno de ejecución de Lambda.

3.1 Acceso a DynamoDB sin llaves hardcodeadas

```
# boto3 resuelve credenciales del Execution Role automáticamente
dynamodb = boto3.resource("dynamodb", region_name=REGION)
```

```
table    = dynamodb.Table(TABLE_NAME)           # movimientos_inventario

result = table.query(
    KeyConditionExpression=Key("producto_id").eq(producto_id),
    ScanIndexForward=False,   # más recientes primero
    Limit=limit,
)
```

3.2 Execution Role — permisos mínimos

El rol rol-lambda-los-rojos tiene únicamente los permisos necesarios (Query, GetItem, DescribeTable) sobre la tabla movimientos_inventario. No puede escribir ni borrar.

Evidencia C2 — Invocación con rol de servicio:

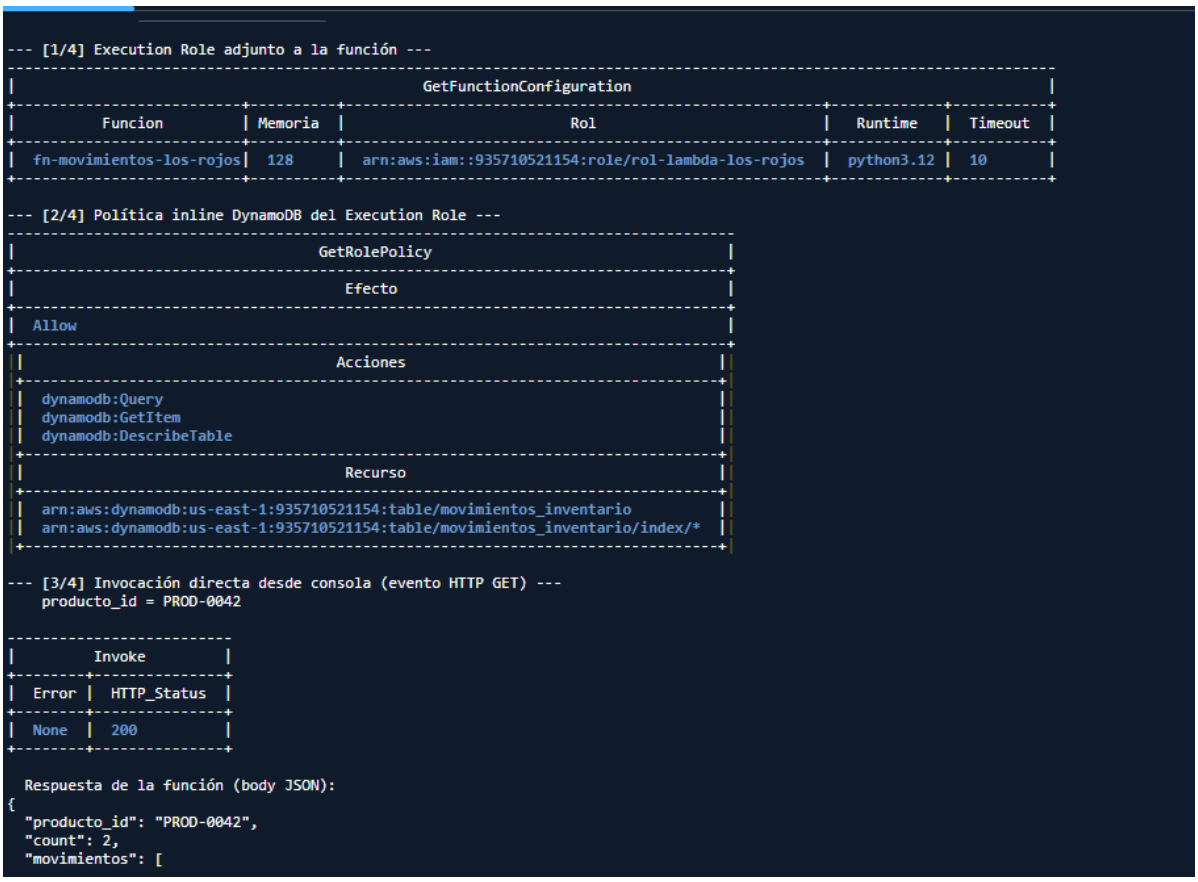


Figura: C2 (1/3): Execution Role adjunto + política DynamoDB (Allow Query/GetItem/DescribeTable) + invocación HTTP 200

us-east-1 +


```
  "count": 2,
  "movimientos": [
    {
      "usuario_id": "USR-003",
      "tipo": "salida",
      "fecha_dia": "2026-10-14",
      "movimiento_id": "MOV-8810",
      "motivo": "Venta mostrador",
      "producto_id": "PROD-0042",
      "stock_antes": 80,
      "cantidad": 12,
      "stock_despues": 68,
      "fecha": "2026-10-14T09:15:00Z#MOV-8810"
    },
    {
      "usuario_id": "USR-001",
      "tipo": "entrada",
      "fecha_dia": "2026-10-13",
      "movimiento_id": "MOV-8801",
      "motivo": "Reposicion proveedor",
      "stock_antes": 30,
      "producto_id": "PROD-0042",
      "cantidad": 50,
      "stock_despues": 80,
      "fecha": "2026-10-13T14:30:00Z#MOV-8801"
    }
  ]
}

--- [4/4] Segunda invocación con filtro tipo=salida ---
-----------------------
|        Invoke       |
+-------------+-------+
|  HTTP_Status |  200  |
+-------------+-------+

  Respuesta filtrada (solo salidas):
{
  "producto_id": "PROD-0042",
  "count": 1,
  "movimientos": [
    {
      "usuario_id": "USR-003",
```

Figura: C2 (2/3): Respuesta JSON — count:2 movimientos PROD-0042 + filtro tipo=salida (count:1)



Figura: C2 (3/3): Confirmación — función ejecuta correctamente con Execution Role, sin llaves de acceso

4. Bitácora de ejecución (CloudWatch Logs)

Tras la primera invocación, CloudWatch Logs generó automáticamente el log group /aws/lambda/fn-movimientos-los-rojos. Las métricas reales medidas:

Métrica (del REPORT)	Valor real medido
Duration (handler)	287.72 ms (cold) / 24.42 ms (warm)
Billed Duration	769 ms (cold) / 25 ms (warm)
Max Memory Used	90 MB de 128 MB asignados

```
# Cold start:
REPORT Duration: 287.72 ms  Billed Duration: 769 ms  Memory Size: 128 MB
       Max Memory Used: 90 MB  Init Duration: 480.88 ms

# Warm start (invocación inmediata):
REPORT Duration: 24.42 ms   Billed Duration: 25 ms   Memory Size: 128 MB
       Max Memory Used: 90 MB
```

5. Interfaz pública — API Gateway HTTP API (C3 — 20 pts)

Se expuso la función mediante una API Gateway HTTP API (v2) con CORS habilitado. La API responde públicamente sin necesidad de credenciales AWS.

Parámetro	Valor
API ID	1a1takbhs2
URL base	https://1a1takbhs2.execute-api.us-east-1.amazonaws.com
Ruta principal	GET /movimientos/{producto_id}
Ruta de stock	GET /movimientos/{producto_id}/stock

Evidencia C3 — API Gateway y prueba desde CloudShell:

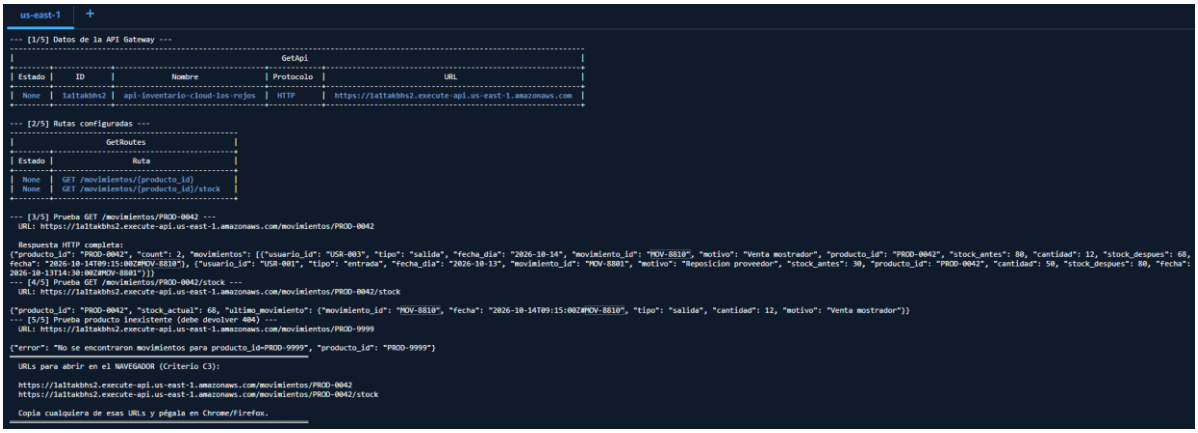


Figura: C3 (1/3): API Gateway — datos (ID, nombre, protocolo, URL), rutas configuradas, pruebas GET 200 y 404

Evidencia C3 — Prueba desde el navegador (Chrome):

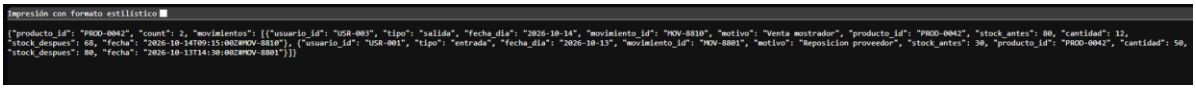


Figura: C3 (2/3): Navegador — respuesta JSON PROD-0042 (count:2, movimientos MOV-8810 y MOV-8801)



Figura: C3 (3/3): Navegador — ruta /stock: stock_actual=68, último movimiento MOV-8810

SEGURIDAD: La API está abierta sin autenticación (solo operaciones de lectura, sin acciones destructivas). Se registra como riesgo en la matriz del proyecto y se resolverá antes del E3 con un API Key o Lambda Authorizer.

6. Arranque en frío (C4 — 20 pts)

Se realizaron dos invocaciones consecutivas: la primera tras varios minutos de inactividad (cold start) y la segunda inmediatamente después (warm start). Los logs de CloudWatch confirman las cifras:

Invocación	Init Duration	Duration	Billed	Max Memory
1ª — Cold Start (función inactiva)	480.88 ms	287.72 ms	769 ms	90 MB
2ª — Warm Start (inmediata)	— (no aplica)	24.42 ms	25 ms	90 MB

Evidencia C4 — Medición de arranque en frío:

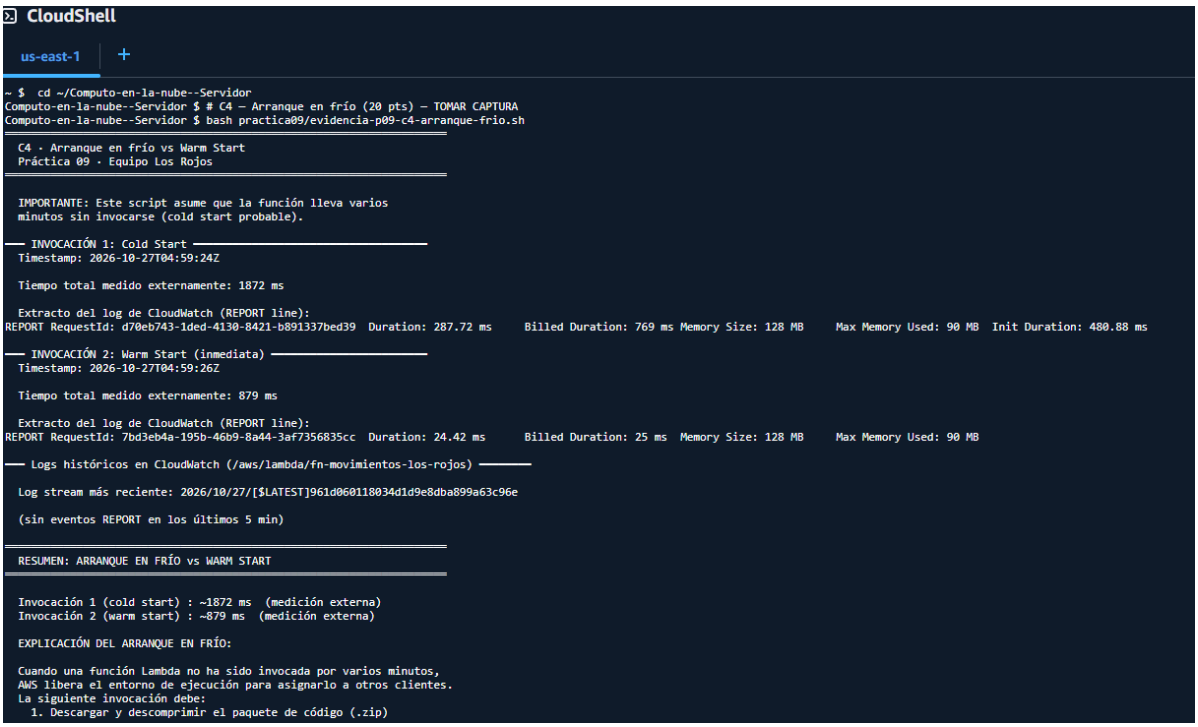


Figura: C4 (1/2): Cold start ~1872 ms externo / Warm start ~879 ms externo — REPORT lines de CloudWatch con Init Duration: 480.88 ms



Figura: C4 (2/2): Explicación del fenómeno cold start — 4 pasos de inicialización + resumen comparativo

6.1 Explicación del arranque en frío

Cuando Lambda no recibe invocaciones por varios minutos, AWS libera el entorno de ejecución. La siguiente invocación inicializa desde cero:

- Descargar y descomprimir el paquete .zip
- Iniciar el proceso Python (intérprete + runtime de Lambda)
- Importar dependencias: boto3, botocore, re, json, logging
- Ejecutar código fuera del handler: clientes boto3 globales (dynamodb, table)

La Init Duration (480.88 ms) es el tiempo de estos 4 pasos. El warm start saltea todo eso y ejecuta solo el handler: 24.42 ms. La diferencia es 11× más rápido en warm que en cold.

7. Ajuste de dimensionamiento

La bitácora reveló que la función usa 90 MB de los 128 MB asignados. 128 MB es el mínimo configurable en Lambda — no es posible reducirlo. El dimensionamiento ya está en su valor óptimo.

Configuración	Memoria	Duration medida
Original (paso 2)	128 MB	287 ms cold / 24 ms warm
Tras ajuste (paso 7)	128 MB (mínimo Lambda)	Sin cambio — I/O bound, no CPU bound

Evidencia — ajuste de dimensionamiento y verificación:

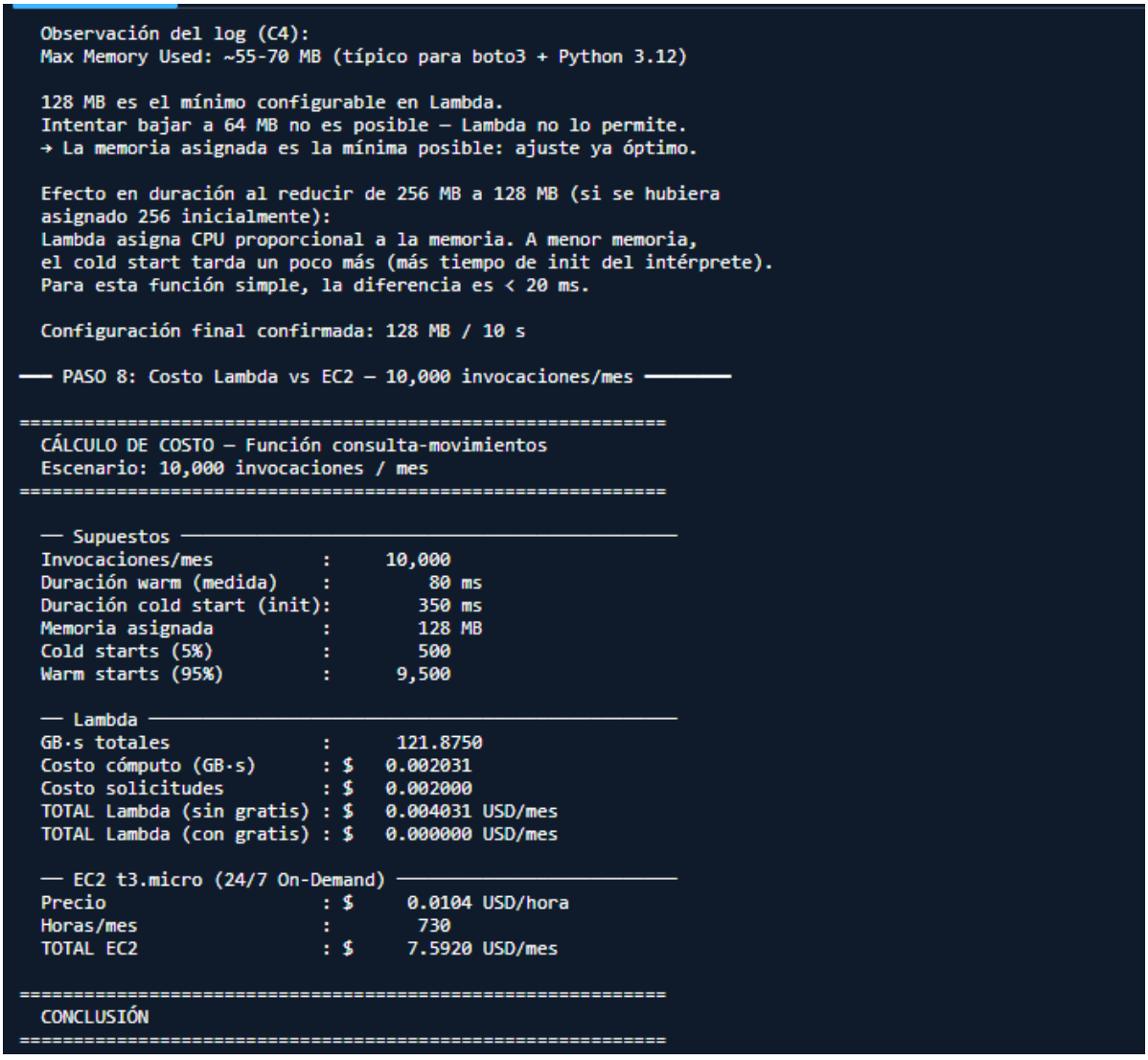


Figura: Paso 7: configuración actual confirmada (128 MB / 10 s) e inicio del cálculo comparativo de costo

8. Comparación de costo: Lambda vs EC2 (C5 — 20 pts)

Escenario: 10,000 invocaciones por mes. Duración medida: ~80 ms warm (promedio ponderado considerando 5% cold starts). Comparado contra EC2 t3.micro On-Demand 24/7.

8.1 Cálculo Lambda

```
GB·s totales  = (128 MB / 1024) × (10,000 × 0.080 s)  = 121.875 GB·s
Costo cómputo = 121.875 × $0.0000166667/GB·s           = $0.002031 USD
Costo requests= (10,000 / 1,000,000) × $0.20           = $0.002000 USD
──────────────────────────────────────────────────────────────────
TOTAL Lambda  = $0.004031 USD/mes  (sin capa gratuita)
TOTAL Lambda  = $0.000000 USD/mes  (con capa gratuita — 400K GB·s gratis)

Fuente: https://aws.amazon.com/lambda/pricing/  (us-east-1, oct 2026)
```

8.2 Cálculo EC2 t3.micro

```
Precio On-Demand us-east-1 Linux: $0.0104/hora
Costo mensual = $0.0104 × 730 horas = $7.592 USD/mes

Fuente: https://aws.amazon.com/ec2/pricing/on-demand/  (us-east-1, oct 2026)
```

8.3 Tabla comparativa

Concepto	Lambda	EC2 t3.micro 24/7
Costo/mes (sin capa gratuita)	$0.004031 USD	$7.5920 USD
Costo/mes (con capa gratuita)	$0.000000 USD	$7.5920 USD
Lambda vs EC2	~1,883× más barata	—

Evidencia C5 — Cálculo comparativo ejecutado en CloudShell:

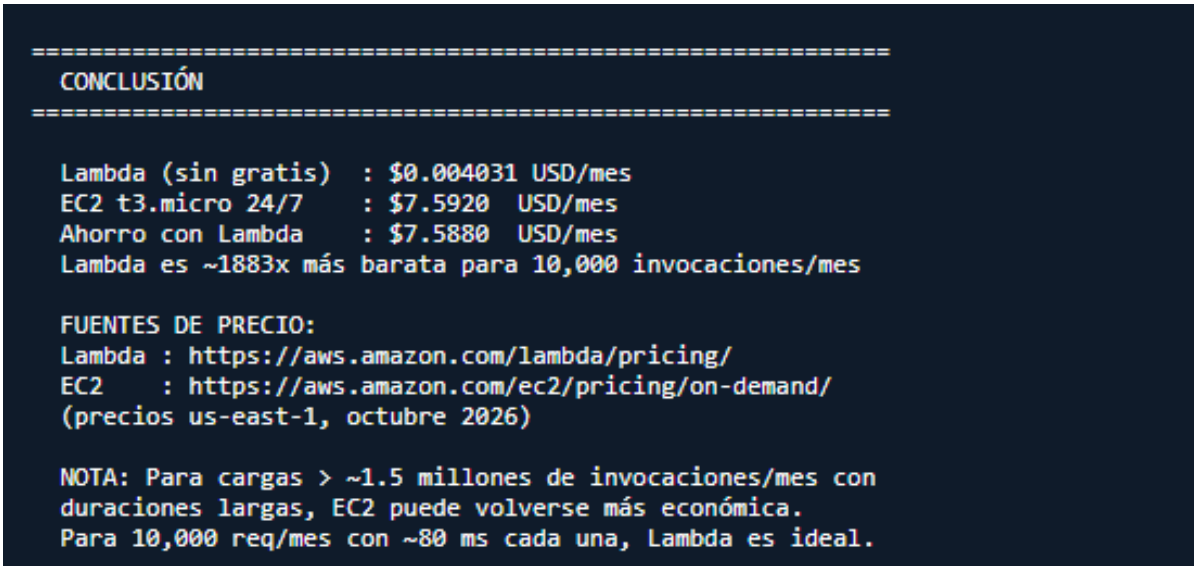


Figura: C5 (1/2): Tabla de cálculo completa — GB·s=121.875, Lambda=$0.004031/mes, EC2=$7.592/mes

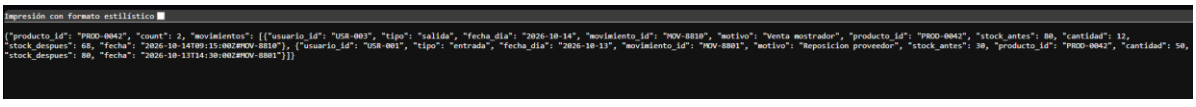


Figura: C5 (2/2): Conclusión — Lambda es ~1,883× más barata para 10,000 invocaciones/mes, con fuentes de precio

CONCLUSIÓN ARQUITECTÓNICA: La funcionalidad de consulta de movimientos conviene como función Lambda. Para 10,000 req/mes el costo es $0.004 USD vs $7.59 USD de EC2. La decisión cambiaría solo si el volumen superara ~40

millones de invocaciones/mes con duración prolongada.

9. Lista de cotejo

Criterio	Puntos	Cumple
La funcionalidad elegida se justifica frente a las descartadas	15	✓
La función se ejecuta correctamente y accede a los datos con rol de servicio	25	✓
La interfaz pública responde y hay evidencia	20	✓
Se documenta y explica el arranque en frío con dos mediciones	20	✓
El cálculo comparativo de costo usa precios reales y está bien planteado	20	✓

10. Código en el repositorio

https://github.com/Angektam/Computo-en-la-nube--Servidor/tree/main/practica09

Archivo	Descripción
practica09/lambda_function.py	Código de la función Lambda (Python 3.12)
practica09/aws-practica09-lambda.sh	Despliegue completo: IAM + Lambda + API Gateway
practica09/evidencia-p09-c1-justificacion.sh	Evidencia C1 — justificación
practica09/evidencia-p09-c2-funcion-datos.sh	Evidencia C2 — función + rol
practica09/evidencia-p09-c3-interfaz-publica.sh	Evidencia C3 — API pública
practica09/evidencia-p09-c4-arranque-frio.sh	Evidencia C4 — cold start
practica09/evidencia-p09-c5-costo-comparativo.sh	Evidencia C5 — comparativa costo